

Week 2: Product and Category Management System

Scenario:

The movie ticket booking company wants to maintain information about movies and their categories. Each movie belongs to a specific category, and the system should manage movie details such as movie name, ticket price, and ticket availability.

1. Design Product and Category Tables

Category Table:

```
SQL> CREATE TABLE CATEGORY(CATEGORY_ID VARCHAR(7),CATEGORY_NAME VARCHAR(20),DESCRIPTION VARCHAR(50));  
Table created.
```

Product Table:

```
SQL> CREATE TABLE PRODUCT(PRODUCT_ID VARCHAR(7),PRODUCT_NAME VARCHAR(20),CATEGORY_ID VARCHAR(7),PRICE NUMBER(9,2),STOCK_QUANTITY NUMBER);  
Table created.
```

The two tables for Category and Product have been created.

Now, we have to find the relationship between the tables and apply constraints to the tables.

2. Define Relationships and Constraints

Relationships:

One Category Can have many Products.

The relationship between Category and Product is One to many (1 → *).

We found that, what is their relationship. Now we have to discuss that which is the key that is going to make this relationship.

Before that, we need to understand about KEY Constraints.

PRIMARY KEY:

The Column(s) or Field(s) that uniquely identifies a row / tuple

In these two tables, which field can be a Primary Key?

In Category Table, Category_ID is a unique Value that helps to identify each category. Likewise, in Product Table Product_ID is a unique Column.

FOREIGN KEY:

The Key which links two tables.

Therefore, Category_ID acts as Foreign Key in the Product Table.

We can explore some other constraints like NOT NULL, CHECK and UNIQUE

NOT NULL:

Ensures that the Column cannot have a empty (null) value. For Example, a Product name or Category name cannot be empty. All category or product must have a name.

CHECK:

Validates the values in a column satisfies the condition. For Example, To check the Price of a Product must always be a positive number.

UNIQUE:

Ensures that there is no repeated value for a column. For Example, If we mention UNIQUE Constraint to Category Name, no two category can have same name.

Defining Relationship and Creating Constraints for Product and Category:

Primary Key:

```
SQL> ALTER TABLE CATEGORY ADD CONSTRAINT PK_CATEGORY PRIMARY KEY (CATEGORY_ID);  
Table altered.  
SQL> ALTER TABLE PRODUCT ADD CONSTRAINT PK_PRODUCT PRIMARY KEY (PRODUCT_ID);  
Table altered.
```

Creating Relationship using FOREIGN KEY:

```
SQL> ALTER TABLE Product ADD CONSTRAINT FK_Product_Category FOREIGN KEY (Category_ID) REFERENCES Category(Category_id);  
Table altered.
```

Adding Constraints:

NOT NULL:

```
SQL> ALTER TABLE Category MODIFY Category_Name VARCHAR(100) NOT NULL;  
Table altered.  
SQL> ALTER TABLE Product MODIFY Product_Name VARCHAR(150) NOT NULL;  
Table altered.
```

CHECK:

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_PRICE CHECK (Price > 0);
Table altered.

SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Stock CHECK(Stock_Quantity >= 0);
Table altered.
```

UNIQUE:

```
SQL> ALTER TABLE Category ADD CONSTRAINT UQ_Category UNIQUE (Category_Name);
Table altered.
```

3. Insert Values:

Category Table:

```
SQL> insert into category values('C001','Action','Action Movies');
1 row created.
```

```
SQL> insert into category values('C002','Comedy','Comedy Movies');
1 row created.
```

```
SQL> insert into category values('C003','Drama','Drama Movies');
1 row created.
```

```
SQL> insert into category values('C004','Romance','Romance Movies');
1 row created.
```

```
SQL> insert into category values('C005','Horror','Horror Movies');
1 row created.
```

```
SQL> insert into category values('C006','Thriller','Thriller Movies');
1 row created.
```

```
SQL> insert into category values('C007','Sci-Fic','Science fiction Movies');
1 row created.
```

```
SQL> insert into category values('C008','Animation','Animation Movies');
1 row created.
```

```
SQL> insert into category values('C009','Adventure','Adventure Movies');
1 row created.
```

```
SQL> insert into category values('C010','Family','Family Movies');  
1 row created.
```

After Insertion Category Table:

CATEGORY_ID	CATEGORY_NAME	DESCRIPTION
C001	Action	Action Movies
C002	Comedy	Comedy Movies
C003	Drama	Drama Movies
C004	Romance	Romance Movies
C005	Horror	Horror Movies
C006	Thriller	Thriller Movies
C007	Sci-Fic	Science fiction Movies
C008	Animation	Animation Movies
C009	Adventure	Adventure Movies
C010	Family	Family Movies

10 rows selected.

Product Table:

```
SQL> insert into product values('P001','Leo','C001',180,50);  
1 row created.
```

```
SQL> insert into product values('P002','Good Night','C002',150,40);  
1 row created.
```

```
SQL> insert into product values('P003','96','C003',160,35);  
1 row created.
```

```
SQL> insert into product values('P004','Love Today','C004',150,45);  
1 row created.
```

```
SQL> insert into product values('P005','Conjuring','C005',180,30);  
1 row created.
```

```
SQL> insert into product values('P006','Vikram','C006',200,50);  
1 row created.
```

```
SQL> insert into product values('P007','Interstellar','C007',220,25);  
1 row created.
```

```
SQL> insert into product values('P008','Toy Story','C008',160,40);
1 row created.
```

```
SQL> insert into product values('P009','Avengers','C009',200,50);
1 row created.
```

```
SQL> insert into product values('P010','The Lion King','C010',170,35);
1 row created.
```

After Insertion Product Table:

PRODUCT_ID	PRODUCT_NAME	CATEGORY_ID	PRICE	STOCK_QUANTITY
P001	Leo	C001	180.00	50
P002	Good Night	C002	150.00	40
P003	96	C003	160.00	35
P004	Love Today	C004	150.00	45
P005	Conjuring	C005	180.00	30
P006	Vikram	C006	200.00	50
P007	Interstellar	C007	220.00	25
P008	Toy Story	C008	160.00	40
P009	Avengers	C009	200.00	50
P010	The Lion King	C010	170.00	35

10 rows selected.

CRUD Operations:

Insert Record:

```
SQL> insert into product values('P011','Master','C001',180,50);
1 row created.
```

Update Price:

```
SQL> UPDATE product
  2  SET price = 200
  3  WHERE product_id = 'P010';

1 row updated.
```

Delete a Product:

```
SQL> delete from product where product_id = 'P011';
1 row deleted.
```

5. Generate Category-wise Product Reports

Write SQL queries to:

- Display all products under each category.

```
SQL> SELECT c.Category_Name,p.Product_Name,p.Price,p.Stock_Quantity FROM Product p JOIN Category c ON p.Category_ID=c.Category_ID;
```

CATEGORY_NAME	PRODUCT_NAME	PRICE	STOCK_QUANTITY
Action	Leo	180.00	50
Comedy	Good Night	150.00	40
Drama	96	160.00	35
Romance	Love Today	150.00	45
Horror	Conjuring	180.00	30
Thriller	Vikram	200.00	50
Sci-Fic	Interstellar	220.00	25
Animation	Toy Story	160.00	40
Adventure	Avengers	200.00	50
Family	The Lion King	200.00	35

10 rows selected.

- Count number of products in each category.

```
SQL> SELECT c.Category_Name,COUNT(p.Product_ID) AS Product_Count FROM Product p JOIN Category c ON p.Category_ID=c.Category_ID GROUP BY c.Category_Name;
```

CATEGORY_NAME	PRODUCT_COUNT
Comedy	1
Animation	1
Family	1
Thriller	1
Drama	1
Romance	1
Horror	1
Adventure	1
Action	1
Sci-Fic	1

10 rows selected.

- Find highest-priced product in each category.

```
SQL> SELECT c.Category_Name,p.Product_Name,p.Price AS Highest_price FROM Product p JOIN Category c ON p.Category_ID=c.Category_ID WHERE p.Price=(SELECT MAX(p2.Price) FROM Product p2 WHERE p2.Category_ID=p.Category_ID);
```

CATEGORY_NAME	PRODUCT_NAME	HIGHEST_PRICE
Action	Leo	180
Comedy	Good Night	150
Drama	96	160
Romance	Love Today	150
Horror	Conjuring	180
Thriller	Vikram	200
Sci-Fic	Interstellar	220
Animation	Toy Story	160
Adventure	Avengers	200
Family	The Lion King	200

10 rows selected.

- Display categories having more than 5 products.

```
SQL> SELECT c.Category_Name FROM Product p JOIN Category c ON p.Category_ID=c.Category_ID GROUP BY c.Category_Name HAVING COUNT(p.Product_ID)>5;
```

no rows selected

- Find average product price category-wise.

CATEGORY_NAME	AVG_PRICE
Comedy	150
Animation	160
Family	200
Thriller	200
Drama	160
Romance	150
Horror	180
Adventure	200
Action	180
Sci-Fic	220

10 rows selected.